Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Dot .Net Core and Progressive Web Application Practical

Module - I

Practical – III

Roll No.:T054	Name:Priti Yadav
Class:TYIT	Batch:2
Date of Assignment: 14/07/26	Date/Time of Submission: 14/07/26

3. Write the program for the following:

a. Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

ALGORITHM:-

1. Start
2. Define a delegate Operation that takes two integers
3. Create a class Calculator
4. Define methods:
   Add(a, b) → print sum
   Subtract(a, b) → print difference
   Multiply(a, b) → print product
   Divide(a, b) → print division
5. In Main() method:
   Create object c of Calculator
6. Declare delegate variable obj
7. Assign and call methods using delegate:
8. Assign All methods and display result
9. Stop

CODE:-

```
using System;
delegate void Operation(int a, int b);
class Calculator{
   public void Add(int a, int b)
   {
      Console.WriteLine("Addition:"+(a+b));
   }
   public void Subtract(int a, int b)
   {
      Console.WriteLine("Subtraction:"+(a-b));
   }
   public void Multiply(int a, int b)
   {
      Console.WriteLine("Multiplication:"+(a*b));
   }
   public void Divide(int a, int b)
   {
         Console.WriteLine("Division:"+(a/b));
   }}
public class Program
{
   public static void Main(string[] args)
   {
      Calculator c= new Calculator();
      Operation obj;
```

```
        obj = c.Add(20, 10);
        obj=c.Subtract(20,10);
        obj=c.Multiply(20, 10);
        obj = c.Divide(20, 10);
        Console.WriteLine("Priti Yadav T054");
    }}
```

OUTPUT:-

```
Addition:30
Subtraction:10
Multiplication:200
Division:2
Priti Yadav T054
```

b. Write a program using multicast delegate.

ALGORITHM:-

1. Start
2. Define a delegate Message with no parameters
3. Create a class Demo
4. Define methods:
   ShowMessage() → print "Hello, Students"
   Learn() → print "Learning C# Delegates"
5. In Main() method:
   Create object d of Demo
6. Declare delegate variable obj
7. Assign ShowMessage method to delegate
   Call delegate → display message
   Assign Learn method to delegate
8. Call delegate → display learning message
9. Stop

CODE:-

```
using System;
delegate void Message();
class Demo{
    public void ShowMessage(){
        Console.WriteLine("Hello,Students");
    }
    public void Learn() {
        Console.WriteLine("Learning C# Delegates");
    }}
public class Program{
    public static void Main(string[] args){
        Demo d = new Demo();
        Message obj;
        obj = d.ShowMessage;
        obj();
        obj = d.Learn;
        obj();
        Console.WriteLine("Priti Yadav T054");
    }}
```

OUTPUT:-

```
Hello,Students
Learning C# Delegates
Priti Yadav T054
```

c. Create a class BankAccount with AccountNumber and Balance.Implement property for Balance with validation (Balance cannot be negative).

ALGORITHM:-

1. Start
2. Create class BankAccount :

Declare property AccountNumber
Declare private variable balance

3. Define property Balance with:
   Get: return balance
   Set: If value ≥ 0 → assign to balance
   Else → print "Balance cannot be negative"
4. In Main() method:
   Create object b of BankAccount
   Set AccountNumber = 12345
   Set Balance = 1000
   Display Account Number
   Display Balance
5. Try setting Balance = -500
6. Display error message
7. Stop

CODE:-

```
using System;
class BankAccount
{
   public int AccountNumber { get; set; }
   private double balance;
   public double Balance
   {
      get
      {
         return balance;
      }
      set
      {
         if (value >= 0)
            balance = value;
         else
            Console.WriteLine("Balance cannot be negative.");
      }  }}
class Program
{
   static void Main(string[] args)
   {
      BankAccount b=new BankAccount();
      b.AccountNumber= 12345;
      b.Balance = 1000;
      Console.WriteLine("Account Number: " + b.AccountNumber);
      Console.WriteLine("Balance: " + b.Balance);
      b.Balance = -500;
      Console.WriteLine("Priti Yadav T054");
   }}
```

OUTPUT:-

```
Account Number: 12345
Balance: 1000
Balance cannot be negative.
Priti Yadav T054
```